📘 Revision Notes: Days 1-4 — AI/ML Fast-Track

Show Image Show Image Show Image Show Image

🎯 **Purpose of this doc:** a fast, dense re-read before you move to Day 5. Every concept has a plain-English explanation + a runnable example + a "why it matters" note. Skim the 🔑 boxes if you're short on time; read everything if you want it to really stick.

🗂️ Table of Contents

🟦 Day 1 — Environment, Python Refresher, First Dataset Look

🔑 Key Idea

Nothing fancy today — just make sure your tools work and you can *load and glance at* real data. This is plumbing, not ML.

1️⃣ Virtual Environments — why bother?

A virtual environment (`venv`) is an isolated Python installation just for this project, so its package versions don't clash with any other project on your machine.

```bash
python3 -m venv venv
source venv/bin/activate    # Mac/Linux
venv\Scripts\activate       # Windows
```

💡 **Extra example:** imagine Project A needs `pandas==1.5` and Project B needs `pandas==2.1`. Without venvs, installing one breaks the other. With venvs, each project has its own private copy — no conflict.

2️⃣ Python Refresher — the 4 building blocks you must be fluent in

Concept	Example	Why it matters for ML
Function + default arg	`def greet(name, excited=False): ...`	You'll write dozens of small helper functions (like `evaluate()` from Day 4)
List comprehension	`[x**2 for x in range(10) if x % 2 == 0]`	Fast way to transform data without verbose loops
Dict comprehension	`{w: len(w) for w in words}`	Used constantly for lookups (e.g. label → index maps)
Class	`class Movie: def __init__(self,...)`	Every ML model object (`LinearRegression()`, etc.) is a class instance

💡 **Extra example — list comprehension in an ML context:**

```
python
ratings = [4.5, 2.0, 3.5, 5.0, 1.5]
good_ratings = [r for r in ratings if r >= 3.5]
# good_ratings = [4.5, 3.5, 5.0]
```

This exact pattern — filter a list by a condition — is what you're doing (at scale, vectorized) every time you filter a DataFrame.

3️⃣ First Dataset Look

```
python
movies = pd.read_csv("movies.csv")
movies.head()     # first 5 rows — sanity check it loaded right
movies.info()     # column names, types, missing-value counts
movies.describe()  # stats: mean, std, min, max for numeric columns
```

Method	Answers the question
`.head()`	"Does this look like what I expect?"
`.info()`	"What types are my columns, and where's data missing?"
`.describe()`	"What's the overall shape/scale of my numbers?"

⚠️ **Common mistake:** skipping `.info()`. This is where you catch that a column you assumed was numeric is actually a string (very common with things like `"$1,000,000"` budget columns) — that will silently break a model later if not caught now.

🟩 Day 2 — Pandas & NumPy: Filtering, GroupBy, Merge

🔑 Key Idea

`NumPy` = fast math on arrays, no loops. `Pandas` = labeled tables built on top of NumPy. `groupby` = **"for each group, compute something."** `merge` = **"join two tables together."** These four operations are 80% of everything you'll do with data for the rest of the roadmap.

1️⃣ NumPy — vectorized operations

```python
import numpy as np
ratings = np.array([4.5, 3.0, 5.0, 2.5, 4.0])

ratings * 2              # [9.0, 6.0, 10.0, 5.0, 8.0] — no loop needed
ratings[ratings >= 4.0]      # [4.5, 5.0, 4.0] — boolean masking
ratings.mean(), ratings.std()   # 3.8, ~0.95
```

💡 **Extra example — why vectorization matters:**

```python
# SLOW way (pure Python loop)
doubled = []
for r in ratings:
  doubled.append(r * 2)

# FAST way (vectorized) — does the same thing, ~50-100x faster on big arrays
doubled = ratings * 2
```

On a 5-element array you won't notice. On a 1-million-row dataset, the loop version can take *minutes* while the vectorized version takes milliseconds.

2️⃣ Filtering (boolean masks)

```python
comedies = movies[movies["genres"].str.contains("Comedy")]
action_or_scifi = movies[
  movies["genres"].str.contains("Action") | movies["genres"].str.contains("Sci-Fi")
]
```

💡 **Extra example — combining 3 conditions:**

```python
# movies that are Comedy AND after 2000 AND rated well
good_recent_comedies = movie_stats[
  (movie_stats["genres"].str.contains("Comedy")) &
  (movie_stats["year"] > 2000) &
  (movie_stats["mean"] >= 4.0)
]
```

Note: use `&` / `|` (not `and`/`or`) and wrap each condition in parentheses — this trips up almost everyone the first time.

3️⃣ GroupBy — "for each X, compute Y"

```python
avg_rating_per_movie = ratings.groupby("movieId")["rating"].mean()
movie_stats = ratings.groupby("movieId")["rating"].agg(["mean", "count", "std"])
```

💡 **Extra example — groupby on a different column, same pattern:**

```python
# average rating given BY each user (are some users just harsher raters?)
avg_rating_per_user = ratings.groupby("userId")["rating"].mean()

# how many movies did each user rate?
ratings_per_user = ratings.groupby("userId")["rating"].count()
```

Same exact mental move as "average rating per movie" — just grouping by a different column. Once `groupby` clicks, you can ask almost any "for each ___, what's the ___" question.

Mental model:

```
groupby("movieId")["rating"].mean()
    ↑            ↑
 "for each movie"    "average this column"
```

4️⃣ Merge — joining two tables

```
python
merged = movie_stats.merge(movies, on="movieId")
```

💡 **Extra example — merge with different column names on each side:**

```
python
# if the key column had different names in each table:
merged = movie_stats.merge(movies, left_on="id", right_on="movieId")
```

Real-world data rarely has perfectly matching column names — knowing `left_on`/`right_on` saves you from renaming columns just to merge them.

5️⃣ `explode()` — splitting one row into many

```
python
genre_list = movies["genres"].str.split("|").explode()
top_genres = genre_list.value_counts().head(10)
```

💡 **Extra example, step by step:**

```
python
# before explode:
# movieId=1, genres="Adventure|Animation|Comedy"   (ONE row)

# after .str.split("|"):
# movieId=1, genres=["Adventure", "Animation", "Comedy"]   (still one row, now a list)

# after .explode():
# movieId=1, genres="Adventure"
# movieId=1, genres="Animation"
# movieId=1, genres="Comedy"      (THREE rows now)
```

This is the single most useful trick for any "pipe-separated" or list-like column — tags, genres, categories, keywords.

🟧 Day 3 — The Core ML Mental Model

🔑 Key Idea

A model learns a pattern from `features → known answer` pairs (**training**), then applies that pattern to new, unseen features (**inference**). You only trust its performance on data it **never saw during training** — that's the entire reason train/test splits exist.

1️⃣ The Flow

```
Features (X)          →  Model  →  Prediction (ŷ)
[budget, runtime, year]  →  [pattern]  →  predicted rating
```

Term	Meaning	Example in our project
Features (X)	The inputs you give the model	`count`, `year`, genre columns
Target (y)	The known answer during training	actual average `rating`
Training	Model adjusts itself using X **and** y	`model.fit(X_train, y_train)`
Inference	Model outputs a guess using X **only**	`model.predict(X_test)`
ŷ ("y-hat")	The model's prediction	`predictions`

2️⃣ Train/Test Split — the most important habit in ML

```
python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

🚨 **Why this exists (the analogy that makes it click):** Imagine a student who studies using the *exact* exam questions and answer key beforehand. They'll score 100% — but that tells you nothing about whether they actually learned the subject. The **test set is a different exam the student never saw.** That's the only honest measure of real understanding.

Split	Model sees the answers?	Purpose
Train (80%)	✅ Yes	Model learns the pattern here
Test (20%)	❌ Never, until final scoring	Honest performance check

💡 **Extra example — what `random_state=42` does:**

```
python
# WITHOUT random_state: every run gives a DIFFERENT random split
# → your MAE would change slightly every time you re-run the same code!

# WITH random_state=42: same split every single time
# → results are reproducible, so you can fairly compare Model A vs Model B
```

The number `42` itself is arbitrary (it's a programming culture joke) — any fixed number works, what matters is that it's *fixed*.

3️⃣ Overfitting vs. Underfitting

	Train error	Test error	What's happening
Underfitting	🔴 High	🔴 High	Model too simple — hasn't learned the real pattern at all
Good fit	🟢 Low	🟢 Low (close to train)	Model learned the real pattern
Overfitting	🟢 Very low	🔴 High	Model memorized training data noise, doesn't generalize

💡 **Extra example — a memorable analogy:**
Underfitting = a student who only studied the chapter titles. Fails both practice questions and the real exam.
Overfitting = a student who memorized the exact practice exam word-for-word, including typos. Aces the practice exam, then fails the real exam because the questions were phrased slightly differently.
Good fit = a student who understood the underlying concepts. Does well on both, because they learned something *transferable*.

🟥 Day 4 — Baselines, Evaluation, Feature Engineering

🔑 Key Idea

A performance number is meaningless in isolation — **"MAE = 0.65" means nothing until you know what a**

dumb guess would score. Always build a trivial baseline first. Then, better features almost always beat a fancier algorithm on the same weak features.

1 The Dummy Baseline

```
python
from sklearn.dummy import DummyRegressor
dummy = DummyRegressor(strategy="mean")  # always predicts the average, ignores features entirely
dummy.fit(X_train, y_train)
```

💡 **Extra example — other dummy strategies:**

```
python
DummyRegressor(strategy="median")    # always predicts the median
DummyClassifier(strategy="most_frequent")  # for classification: always predicts the majority class
DummyClassifier(strategy="stratified")    # predicts randomly, matching class proportions
```

For classification tasks especially, `most_frequent` is a crucial sanity check — if 95% of your data is one class, a model that just always guesses that class gets "95% accuracy" while being completely useless. This is exactly why you'll add precision/recall later (Day 7).

2 Evaluation Metrics — what each one actually tells you

Metric	Formula (intuition)	Speaks in	Good for
MAE (Mean Absolute Error)	average of `|actual - predicted|`	Same units as target (e.g. "stars")	Easy to explain to anyone: "off by X on average"
RMSE (Root Mean Squared Error)	like MAE but squares errors first	Same units as target	Punishes big misses harder than small ones
R^2	fraction of variance explained	0 to 1 (can go negative)	One number to track "is my model even useful"

💡 **Extra worked example:** Actual ratings: `[4.0, 3.0, 5.0]` — Predicted: `[3.5, 3.0, 4.0]`
Errors: `[0.5, 0.0, 1.0]`
MAE = (0.5 + 0.0 + 1.0) / 3 = **0.5** → "on average, off by half a star"
RMSE = √((0.25 + 0.0 + 1.0) / 3) = √0.417 = **0.65** → higher than MAE because the 1.0 error got squared (weighted more)
RMSE > MAE always (or equal) — the gap between them tells you if you have a few big outlier errors (large gap) or consistently-sized errors (small gap).

3️⃣ Feature Engineering — One-Hot Encoding Genres

```
python
genre_dummies = movie_stats["genres"].str.get_dummies(sep="|")
```

💡 **Extra example, visualized:**

```
Before:
title          genres
Toy Story      Adventure|Animation|Comedy

After get_dummies(sep="|"):
title     Adventure  Animation  Comedy  Drama  ...
Toy Story     1          1        1       0    ...
```

Each genre becomes its own 0/1 column. This is **one-hot encoding** — turning a categorical (text) feature into numbers a model can actually do math with. You'll use this exact pattern constantly, not just for genres — for any categorical column (country, platform, director, etc.).

⚠️ **Why not just number-code genres (Comedy=1, Drama=2, Action=3)?** Because that implies a false order/distance — it tells the model "Action (3) is 3x more than Comedy (1)," which is meaningless. One-hot encoding avoids inventing a fake relationship between categories.

4️⃣ Reading Model Coefficients

```
python
coefs = pd.Series(lr_genres.coef_, index=X_full.columns).sort_values(ascending=False)
```

💡 **Extra example — how to read a coefficient:** If `coefs["Documentary"] = 0.35`, that means: *holding all other features constant, being tagged "Documentary" is associated with a predicted rating about 0.35 stars higher.* If `coefs["Horror"] = -0.20`, Horror is associated with about 0.20 stars lower.
This is **correlation the model found, not proof of causation** — Documentaries aren't inherently "better movies," they're likely rated by a smaller, more self-selected audience of people who chose to watch a documentary. Always read coefficients with that caveat.

5️⃣ Building a Results Table (habit to keep for the rest of the roadmap)

```
python
results_df = pd.DataFrame(results)
```

name	mae	rmse	r2
Dummy (mean)	0.62	0.78	0.00
LinearRegression (count+year)	0.58	0.74	0.09
LinearRegression (+genres)	0.49	0.63	0.34

🔑 **This table is the whole point of Day 4.** Notice the pattern: adding genre features moved R^2 from 0.09 → 0.34 — a much bigger jump than switching algorithms would give you on weak features. **Better features usually beat fancier models.** You'll re-learn this lesson in every phase of the roadmap.

🧠 Glossary — Every Term So Far

Term	One-line definition
venv	Isolated Python environment so package versions don't clash across projects
DataFrame	Pandas' table structure — rows + labeled columns
Vectorization	Doing math on a whole array at once instead of looping
Boolean mask	Filtering rows using a `True`/`False` condition
GroupBy	"For each group, compute this"
Merge	Joining two tables on a shared key column
Explode	Turning one row with a list into multiple rows
Feature (X)	An input the model uses to make predictions
Target (y)	The known answer the model is trying to predict
Train/Test Split	Holding back data the model never trains on, for honest evaluation

Term	One-line definition
Overfitting	Model memorized training noise, fails on new data
Underfitting	Model too simple, fails on both training and new data
Baseline	A trivial model (e.g. "always guess the average") used as a sanity-check floor
MAE	Average absolute prediction error, in the target's own units
RMSE	Like MAE but penalizes large errors more
R^2	Fraction of variance in the target explained by the model (0-1, can go negative)
One-hot encoding	Turning a categorical column into multiple 0/1 columns
Coefficient	In linear models, how much a feature pushes the prediction up/down

✅ Self-Test Quiz (no peeking until you try)

Why do we need a virtual environment per project instead of installing packages globally?
What's the difference between `.head()`, `.info()`, and `.describe()` — when would you use each?
Write the `groupby` line that gives the **median** rating per user (not mean).
Why does filtering need `&`/`|` instead of `and`/`or`, and why do the conditions need parentheses?
What does `random_state=42` actually control, and why does it matter for fair model comparison?
A model gets 0.02 train error and 0.45 test error. Overfitting or underfitting? How do you know?
Your dummy baseline MAE is 0.62. Your real model gets MAE 0.60. Is that model actually useful? What else would you want to know?
Why is R^2 a more "at-a-glance" useful metric than MAE when comparing many models in a table?
Why not encode genres as `Comedy=1, Drama=2, Action=3` instead of one-hot encoding?
If a genre's coefficient is negative, does that prove the genre causes lower ratings? Why or why not?

<details> <summary>💡 Click to reveal answers</summary>

Prevents version conflicts between projects (e.g. one needs pandas 1.5, another needs 2.1).
`.head()` = quick visual sanity check of the data; `.info()` = types + missing values; `.describe()` = numeric distribution/scale.
`ratings.groupby("userId")["rating"].median()`
Python's `and`/`or` work on single booleans, not element-wise on arrays; `&`/`|` are the vectorized (elementwise) versions. Parentheses are needed because of operator precedence — without them, `&` binds tighter than comparison operators and breaks the expression.

It fixes which rows go into train vs. test, so re-running the code gives an identical split every time — necessary for fairly comparing models against each other.
Overfitting — huge gap between train (very low) and test (high) error means the model memorized training data instead of learning a generalizable pattern.
Barely — only marginally better than guessing the average, so it's not very useful yet. You'd want to see the R^2 too, and think about whether better features (not a fancier algorithm) would help more.
R^2 is bounded and interpretable (0 = useless, 1 = perfect) regardless of the target's scale, so you can compare across different targets/models at a glance without knowing the units.
It implies a false numeric relationship/order (Action isn't "3x" Comedy) that could mislead a model doing math with those numbers.
No — it's a correlation the model found in the data, not proof of causation. There could be confounding factors (e.g. who chooses to watch/rate that genre).

</details>

🚀 **Next up:** Day 5 — more feature engineering, watching your results table grow, on the way to Day 13's Project 1 ship date.